

#46

MILLION DICE ROLL STATISTICS SIMULATOR



When you roll two six-sided dice, there's a 17 percent chance you'll roll a 7. That's much better than the odds of rolling a 2: just 3 percent. That's because there's only one combination of dice rolls that gives you 2 (the one that occurs when both dice roll a 1), but many combinations add up to seven: 1 and 6, 2 and 5, 3 and 4, and so on.

But what about when you roll three dice? Or four? Or 1,000? You could mathematically calculate the theoretical probabilities, or you can have the computer roll a number of dice one million times to empirically figure them out. This program takes that latter approach. In this program, you tell the computer to roll N dice one million times and remember the results. It then displays the percentage chance of each sum.

This program does a massive amount of computation, but the computation itself isn't hard to understand.

The Program in Action

When you run *milliondicestats.py*, the output will look like this:

```
Million Dice Roll Statistics Simulator
By Al Sweigart al@inventwithpython.com

Enter how many six-sided dice you want to roll:
> 2
Simulating 1,000,000 rolls of 2 dice...
36.2% done...
73.4% done...
TOTAL - ROLLS - PERCENTAGE
 2 - 27590 rolls - 2.8%
 3 - 55730 rolls - 5.6%
 4 - 83517 rolls - 8.4%
 5 - 111526 rolls - 11.2%
 6 - 139015 rolls - 13.9%
 7 - 166327 rolls - 16.6%
 8 - 139477 rolls - 13.9%
 9 - 110268 rolls - 11.0%
10 - 83272 rolls - 8.3%
11 - 55255 rolls - 5.5%
12 - 28023 rolls - 2.8%
```

How It Works

We simulate the roll of a single six-sided die by calling `random.randint(1, 6)` on line 30. This returns a random number between 1 and 6, which gets added to the running total for however many dice are rolled together. The `random.randint()` function has a uniform distribution, meaning each number is just as likely as any other to be returned.

The program stores the results of this roll with the results dictionary. The keys to this dictionary are each possible dice roll total, and the values are how many times this total has been encountered. To get the frequency percentage, we divide the number of times a total has been encountered by 1,000,000 (the number of dice rolls in this simulation) and multiply it by 100 (to get a percentage between 0.0 and 100.0 instead of 0.0 and 1.0). By doing some algebra, we can figure out that this is the same as dividing the number of encounters by 10,000, which we do on line 37.

```
1. """Million Dice Roll Statistics Simulator
2. By Al Sweigart al@inventwithpython.com
3. A simulation of one million dice rolls.
4. View this code at https://nostarch.com/big-book-small-python-projects
5. Tags: tiny, beginner, math, simulation"""
6.
7. import random, time
8.
9. print('Million Dice Roll Statistics Simulator
10. By Al Sweigart al@inventwithpython.com
11.
```

```

12. Enter how many six-sided dice you want to roll:''')
13. numberOfDice = int(input('> '))
14.
15. # Set up a dictionary to store the results of each dice roll:
16. results = {}
17. for i in range(numberOfDice, (numberOfDice * 6) + 1):
18.     results[i] = 0
19.
20. # Simulate dice rolls:
21. print('Simulating 1,000,000 rolls of {} dice...'.format(numberOfDice))
22. lastPrintTime = time.time()
23. for i in range(1000000):
24.     if time.time() > lastPrintTime + 1:
25.         print('{}% done...'.format(round(i / 10000, 1)))
26.         lastPrintTime = time.time()
27.
28.     total = 0
29.     for j in range(numberOfDice):
30.         total = total + random.randint(1, 6)
31.     results[total] = results[total] + 1
32.
33. # Display results:
34. print('TOTAL - ROLLS - PERCENTAGE')
35. for i in range(numberOfDice, (numberOfDice * 6) + 1):
36.     roll = results[i]
37.     percentage = round(results[i] / 10000, 1)
38.     print(' {} - {} rolls - {}'.format(i, roll, percentage))

```

After entering the source code and running it a few times, try making experimental changes to it. On your own, you can also try to figure out how to do the following:

- Try rolling 8-, 10-, 12-, or 20-sided dice.
- Try simulating two-sided coin tosses.

Exploring the Program

Try to find the answers to the following questions. Experiment with some modifications to the code and rerun the program to see what effect the changes have.

1. What happens if you change `lastPrintTime + 1` on line 24 to `lastPrintTime + 2`?
2. What bug do you cause if you delete or comment out `results[total] = results[total] + 1` on line 31?
3. What error happens if the user types letters instead of a number for the number of six-sided dice to roll?